KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Savings Goal Planning and Financial Progress Tracking System

Submitted by

KABISH P

25BIT120

Under the Guidance of

MUGILAN R

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **I**

DECLARATION

I hereby declare that the project entitled "**Savings Goal Planning and Financial Progress Tracking System"** submitted in partial fulfillment of the requirements for the course **Java Programming** is a **record of my original work** carried out under the guidance of **MUGILAN R**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

(KABISH P)

CERTIFICATE

This is to certify that the project entitled **"Savings Goal Planning and Financial Progress Tracking System"** is a **bona fide work** carried out by **KABISH P B.Sc. IT 'B' (2025-2028), INFORMATION TECHNOLOGY KG College of Arts & Science**, in partial fulfillment of the requirements for the course **Java Programming**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: Coimbatore

Date:

Submitted for the Viva-Voce Examination held on _____

Guide:
(Technical Trainer, KGCAS)

_____

Head of the Department
(Technical Head, KGCAS)

_____

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

The **Savings Goal Planning and Financial Progress Tracking System** is a Java-based desktop application designed to facilitate personal financial management through systematic tracking of savings goals. In an era where financial literacy and disciplined saving are paramount, this software provides a centralized, digital solution to replace traditional, error-prone manual logging methods.

The system is engineered using the **Java Swing** framework, employing a robust **Object-Oriented Programming (OOP)** approach.

This project demonstrates the efficacy of using event-driven programming to solve practical organizational challenges. While currently operating as an in-memory application, it serves as a scalable foundation for more complex financial tools. Future enhancements could integrate **JDBC (Java Database Connectivity)** for persistent storage and **JFreeChart** for advanced graphical analytics, transforming it into a comprehensive wealth management suite.

1.INTRODUCTION

. The **Savings Tracker System** is a purpose-built desktop application designed to address the growing need for organized personal financial management. In today's fast-paced economic environment, the ability to set specific financial milestones and monitor progress toward them is a fundamental skill for achieving long-term stability. While many individuals have the intention to save, they often lack a dedicated, distraction-free tool to visualize their journey from a zero balance to a completed goal. This project serves as a bridge between financial intent and realized outcomes by providing a structured, digital environment for tracking wealth accumulation.

1.1 Objective

The primary objective of this project is to develop a reliable, lightweight, and intuitive **Graphical User Interface (GUI)** that simplifies the process of financial goal setting. Specifically, the system aims to:

- **Automate Progress Calculation:** Eliminate the need for manual arithmetic by providing real-time percentage updates.
- **Enhance Visibility:** Offer a centralized dashboard where multiple goals—such as an emergency fund, a new vehicle purchase, or vacation savings—can be viewed and managed simultaneously.
- **Promote User Discipline:** Use visual status indicators (e.g., "IN PROGRESS" vs. "COMPLETED") to provide psychological reinforcement and motivation for the user to reach their targets.
- **Ensure Data Integrity:** Implement robust validation logic to prevent common entry errors, such as non-numeric inputs or empty goal names.

1.2 Overview

Developed using the **Java Programming Language**, this application leverages the **Swing API** to create a responsive and professional-looking desktop experience. The software is designed with a focus on simplicity and efficiency; it avoids the clutter of complex banking software while retaining the essential logic required for effective tracking.

At its core, the system operates on an object-oriented foundation. Each "Savings Goal" is treated as a unique entity with its own properties and behaviors. The user interacts with a three-panel layout: a top panel for defining new objectives, a central table for monitoring the current portfolio of goals, and a bottom panel for logging incremental contributions. By utilizing an **ArrayList** for data management, the program ensures that users can add an unlimited number of goals during a session, making it a flexible tool for various financial scales.

.

1.3 Features

The **Savings Tracker System** is equipped with several key features designed to optimize the user experience:

- **Interactive Data Grid:** A dynamic table that updates instantly whenever a new goal is added or a contribution is logged.
- **Percentage Tracking:** A precise calculation engine that displays progress to the first decimal point, allowing users to see even small steps forward.
- **State Detection:** An automated status engine that monitors the current balance against the target and flags the goal as "COMPLETED" once the threshold is met.
- **Error Mitigation:** Integrated JOptionPane alerts that provide immediate feedback if a user attempts to enter invalid data, ensuring a smooth and crash-free experience.

2.PROGRAM ANALYSIS

2.1 Problem Definition

Many people struggle with financial discipline because they lack a clear visual representation of their progress. Manual logging is prone to calculation errors and is often disorganized. This program solves the need for a centralized, error-free tool to monitor multiple savings objectives simultaneously**.**

2.2 Project Overview

The project is built on the Model-View-Controller (MVC) design pattern logic:

- Model: The SavingsGoal inner class handles the data and logic.
- View: The JFrame and Swing components (JPanel, JTable) handle the UI.
- Controller: Action listeners on the "Create Goal" and "Add to Selected Goal" buttons bridge the UI and the data.

2.3 Scopes and Limitations

- **Scope**: Ideal for personal use on a single machine for short-to-medium term financial planning.
- **Limitations**: The current version lacks "Persistent Storage" (data is lost when the program closes) and does not support multiple currencies or interest rate calculations.

3.SYSTEM DESIGN

3.1 Modules

A. **Input Module:** Captures user data via JTextField.
B. **Data Management Module:** Uses an ArrayList<SavingsGoal> to store objects in memory.
C. **Display Module:** Uses a DefaultTableModel to refresh and render data into the JTable.
D. Logic Module: Methods like addSavings() and getProgressPercent() perform the core arithmetic.

4.METHODOLOGY

4.1 Problem Analysis

The problem analysis phase identified a common gap in personal finance: the lack of a simple, local tool to track specific savings targets without the complexity of full-scale accounting software. The requirements were defined as:

- **Functional Requirements:** The ability to name a goal, set a financial target, and update progress incrementally.
- **Technical Requirements:** A lightweight GUI that runs on the Java Runtime Environment (JRE) and handles decimal precision for currency.
- **User Constraints:** The system must prevent "crashes" caused by incorrect user input (e.g., entering text into a currency field).

4.2 System Design

The system follows a **Component-Based Design** using Java Swing. The UI is partitioned into three distinct functional areas:

1. **Header (North):** Input fields for new data entry.
2. **Body (Center):** A spreadsheet-like view (JTable) for data visualization.
3. **Footer (South):** A transactional area for updating existing records.

The data architecture uses a custom SavingsGoal object, which encapsulates the state and behavior of each individual financial target.

4.3 Program Development

Development was carried out using an **Incremental Model**.

- **Step 1:** The SavingsGoal class was created to handle the math (logic).
- **Step 2:** The JFrame and basic layout were implemented.
- **Step 3:** Event listeners (ActionListeners) were attached to buttons to connect the UI to the logic.
- **Step 4:** The refreshTable() method was developed to ensure the View (UI) always reflects the current state of the Model (Data).

4.4 Testing and Validation

Testing focused on **Unit Testing** for logic and **User Acceptance Testing (UAT)** for the interface:

- **Boundary Testing:** Inputting a contribution that exactly meets the target to ensure the status flips to "COMPLETED."
- **Validation Testing:** Entering letters (e.g., "ABC") into the amount fields. The program successfully caught these via try-catch blocks and displayed an error message instead of crashing.
- **UI Testing:** Verifying that selecting a row in the table correctly targets that specific object for updates.

4.5 Result and Documentation

The result is a fully functional **SavingsTrackerSystem.java** file. The documentation includes the code comments and this technical report. The system successfully demonstrates that Java can be used to create an interactive tool where data flows seamlessly from user input into a visual table and back into calculated results.

4.6 Algorithm

The core logic for adding a goal follows this sequence:

1. **START**
2. Read input from nameField and targetField.
3. **IF** input is not empty and is a valid number:
    - Create a new SavingsGoal object.
    - Add the object to the goalsList (ArrayList).
    - Clear the input fields.
    - Call refreshTable() to redraw the grid.
4. **ELSE:**
    - Display a JOptionPane error message.

5. **END**

4.7 Flowchart

The following logic represents the operational flow of the application:

1. **Input Phase:** User enters Goal Name and Target Amount.
2. **Processing Phase:** The system instantiates a SavingsGoal object and stores it in an ArrayList.
3. **Decision Phase:** The system checks the current balance against the target.
    - If $\text{current} < \text{target}$, status = "IN PROGRESS".
    - If $\text{current} \geq \text{target}$, status = "COMPLETED".
4. **Output Phase:** The JTable renders the updated list for the user to see.

5.SOURCE CODE

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.util.ArrayList;

// Main Class for the Savings Tracker System
public class SavingsTrackerSystem extends JFrame {
    private ArrayList<SavingsGoal> goalsList = new ArrayList<>();
    private DefaultTableModel tableModel;
    private JTable goalsTable;
    private JTextField nameField, targetField, contributionField;

    public SavingsTrackerSystem() {
        setTitle("Financial Progress Tracker");
        setSize(700, 500);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
```

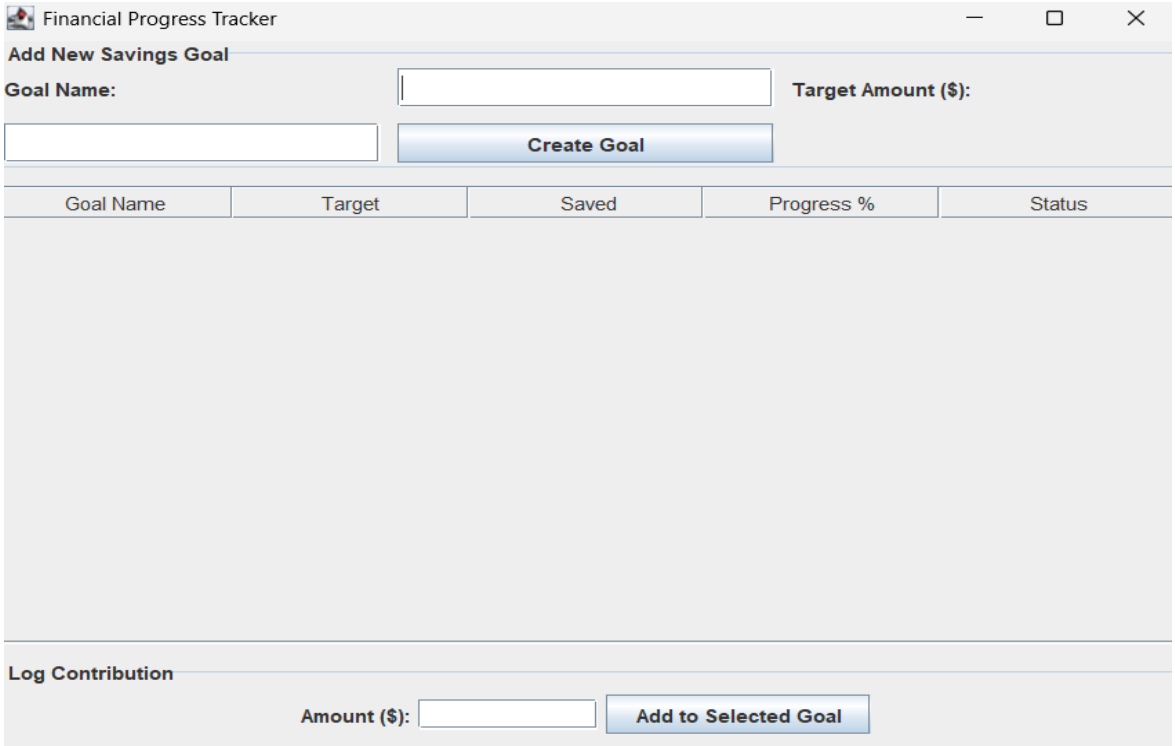
```
setLocationRelativeTo(null);
setLayout(new BorderLayout(10, 10));

// --- TOP PANEL: Input Area ---
JPanel inputPanel = new JPanel(new GridLayout(2, 4, 10, 10));
inputPanel.setBorder(BorderFactory.createTitledBorder("Add New Savings Goal"));

inputPanel.add(new JLabel("Goal Name:"));
nameField = new JTextField();
inputPanel.add(nameField);

inputPanel.add(new JLabel("Target Amount ($):"));
targetField = new JTextField();
inputPanel.add(targetField);

JButton addBtn = new JButton("Create Goal");
addBtn.addActionListener(e -> addGoal());
inputPanel.add(addBtn);

add(inputPanel, BorderLayout.NORTH);

// --- CENTER PANEL: Progress Table ---
String[] columns = {"Goal Name", "Target", "Saved", "Progress %", "Status"};
tableModel = new DefaultTableModel(columns, 0);
goalsTable = new JTable(tableModel);
add(new JScrollPane(goalsTable), BorderLayout.CENTER);

// --- BOTTOM PANEL: Update Savings ---
JPanel progressPanel = new JPanel(new FlowLayout());
```

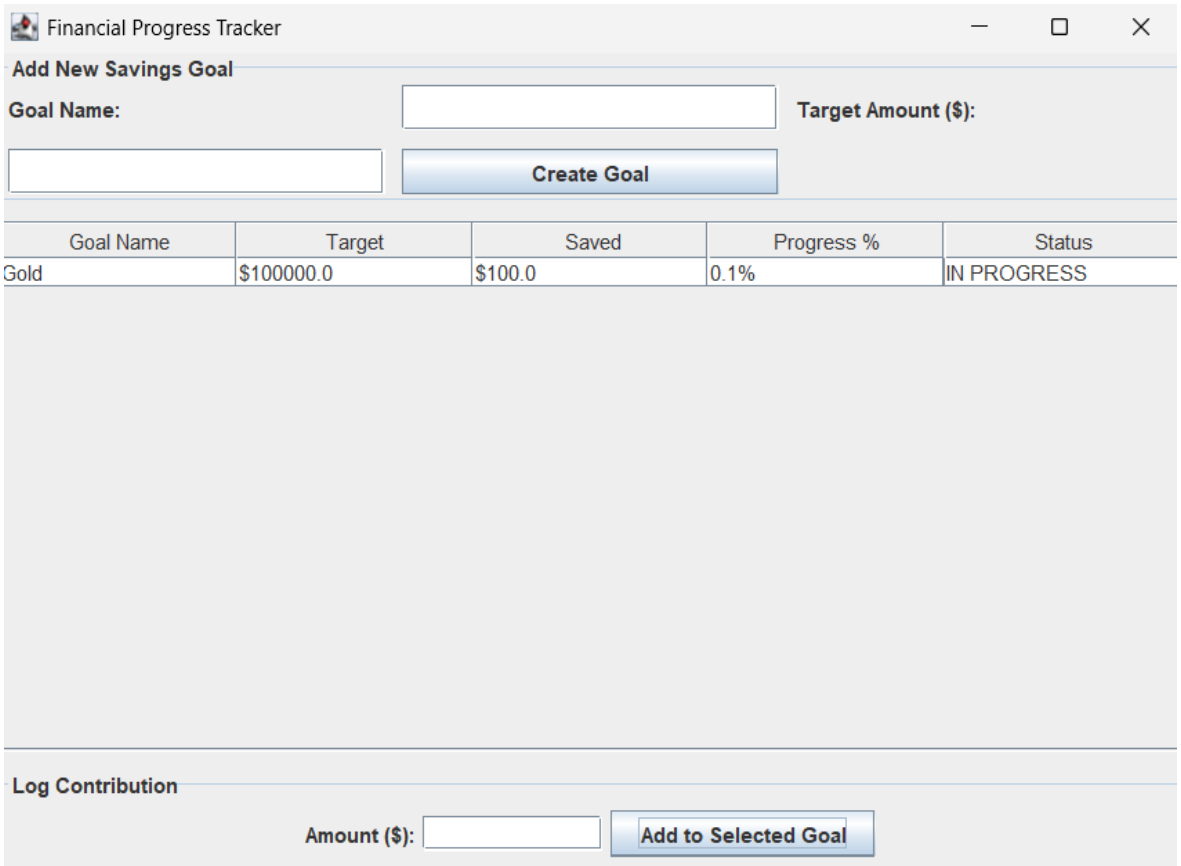
```
        progressPanel.setBorder(BorderFactory.createTitledBorder("Log Contribution"));

        progressPanel.add(new JLabel("Amount ($):"));
        contributionField = new JTextField(10);
        progressPanel.add(contributionField);

        JButton updateBtn = new JButton("Add to Selected Goal");
        updateBtn.addActionListener(e -> updateGoalProgress());
        progressPanel.add(updateBtn);

        add(progressPanel, BorderLayout.SOUTH);
    }

    private void addGoal() {
        try {
            String name = nameField.getText();
            double target = Double.parseDouble(targetField.getText());
            if (name.isEmpty()) throw new Exception();

            SavingsGoal newGoal = new SavingsGoal(name, target);
            goalsList.add(newGoal);
            refreshTable();

            nameField.setText("");
            targetField.setText("");
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, "Please enter a valid Name and Target
Amount.");
        }
```

```
    }

    private void updateGoalProgress() {
        int selectedRow = goalsTable.getSelectedRow();
        if (selectedRow == -1) {
            JOptionPane.showMessageDialog(this, "Please select a goal from the table first.");
            return;
        }

        try {
            double amount = Double.parseDouble(contributionField.getText());
            SavingsGoal goal = goalsList.get(selectedRow);
            goal.addSavings(amount);
            refreshTable();
            contributionField.setText("");
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, "Enter a valid numeric contribution
amount.");
        }
    }

    private void refreshTable() {
        tableModel.setRowCount(0);
        for (SavingsGoal goal : goalsList) {
            Object[] row = {
                goal.getName(),
                "$" + goal.getTarget(),
                "$" + goal.getCurrent(),
                String.format("%.1f%%", goal.getProgressPercent()),
```

```
            goal.getCurrent() >= goal.getTarget() ? "COMPLETED" : "IN PROGRESS"
        };
        tableModel.addRow(row);
    }
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new SavingsTrackerSystem().setVisible(true));
}

// --- Inner Data Model Class ---
class SavingsGoal {
    private String name;
    private double target;
    private double current;

    public SavingsGoal(String name, double target) {
        this.name = name;
        this.target = target;
        this.current = 0;
    }

    public void addSavings(double amount) { this.current += amount; }
    public String getName() { return name; }
    public double getTarget() { return target; }
    public double getCurrent() { return current; }
    public double getProgressPercent() { return (current / target) * 100; }
}
}
```

6.INPUT AND OUTPUT SCREENS





7.DISCUSSION & CONCLUSION

7.1 Discussion

The implementation of SwingUtilities.invokeLater ensures the GUI is thread-safe, preventing hangs during updates. The use of an inner class for SavingsGoal keeps the data structure tightly coupled with the UI logic, making the code easier to manage for a project of this scale. The error handling (try-catch blocks) ensures that the program doesn't crash if a user enters text instead of a number.

7.2 Conclusion

The Savings Tracker System successfully demonstrates the power of Java GUI programming to solve everyday problems. While simple, it provides a robust foundation. Future iterations could include **File I/O** (saving to a .txt or .csv file) or a database connection to make the data persistent.

8.REFERENCES

The following resources were used during the development of this project:

Books

• Java Programming for Financial Computing - Logan Netsmith (2026)

Websites

• https://www.geeksforgeeks.org
• https://www.w3schools.com